

MonitorBaseClaude Human System Guide

A readable memory document for how the WinForms monitor, Monitor MCP tools, Roslyn CodeLens MCP tools, SQLite solution index, staging workflow, and review gate fit together.

Repository: `C:\VSCodeProjects\MonitorBaseClaude`

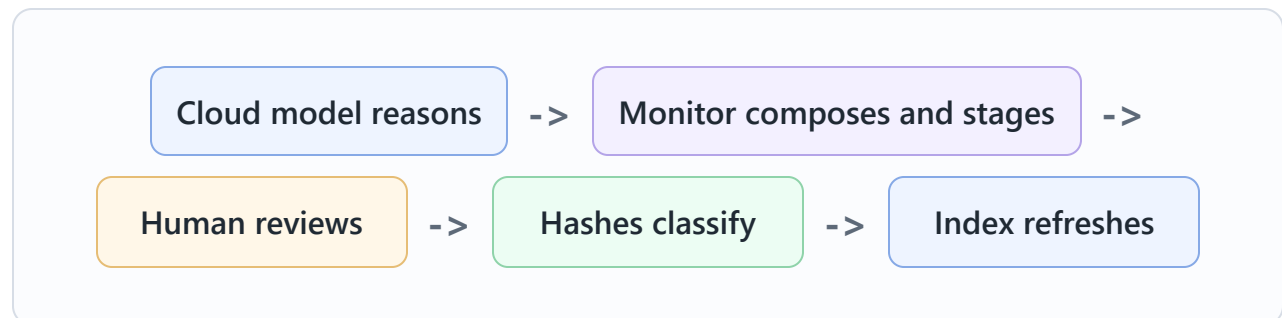
Purpose: a human reference, not an agent instruction file.

Executive Summary

MonitorBaseClaude lets a cloud model reason over a local compiler-backed map, compose edits into a monitor-owned Working mirror, and change watched source only after human diff review and vote-plus-hash verification.

Core idea: the model can reason and propose, but local tools own state, staging, validation, review records, and hash classification.

Trust hierarchy: `dotnet build` is truth, overlay compile is the safety floor, and the solution index is the fast navigation layer. The index does not need to model every MSBuild detail to be valuable, but it must be explicit when it is incomplete.



Main Components

WinForms App

Human/operator UI,
MCP hub visibility,
Solution Index viewer,
review and test surfaces.

Monitor MCP Server

Safe workflow API for
file reads, source maps,
candidate composition,
staged review, and
review decisions.

Roslyn CodeLens MCP

Live compiler-style
semantic navigation:
diagnostics, references,
callers, implementations,
type overview, and call
graph.

Solution Index

SQLite
symbol/reference/caller/relat
index built from Roslyn
semantic APIs over
watched C# source.

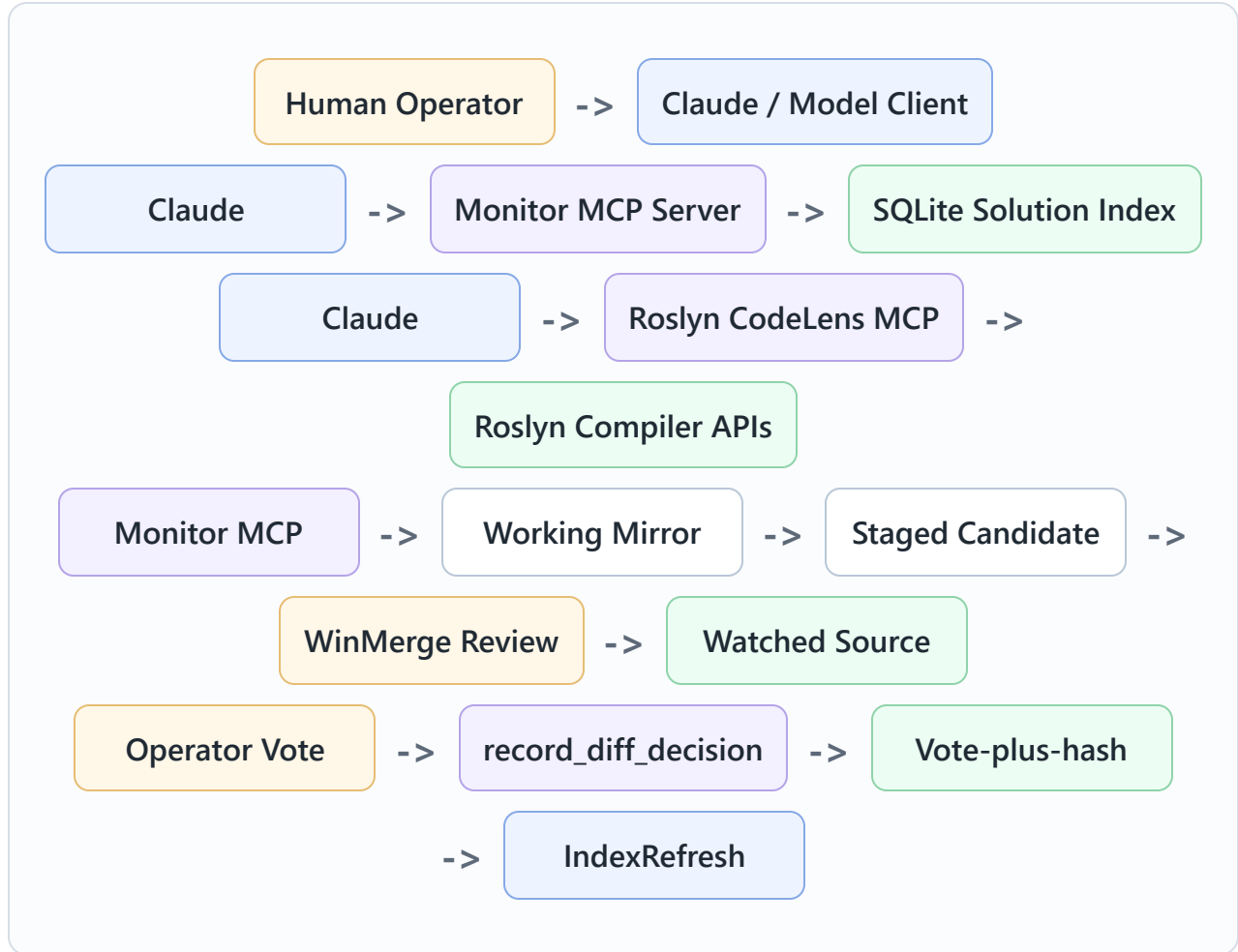
Working Mirror

Monitor-owned
candidate files, staged
snapshots, sessions,
history, ledgers, and
index databases.

WinMerge / Host Review

The actual human
save/no-save gate for
watched source. Accept
all or reject all.

Architecture Diagram



Source Boundaries

World	Examples	Mutation Rule
Product / monitor source	<code>MonitorBaseClaude</code> , <code>MonitorBaseClaude.McpServer</code> , docs, tests	Codex can edit this repository normally.
Watched project source	DBV2, WebViewer, or whatever solution is configured as observed	Claude/model edits must go through Monitor staging and human review.
Monitor-owned generated state	<code>Working\</code> , indexes, history, sessions, staged snapshots	Tools own this. It is not product source and not watched project source.

The Solution Index

The solution index is the local map Claude should ask for before it reads file bodies. It is built from Roslyn semantic APIs over watched C# source and persisted as SQLite.

```
Working\Indexes\<observedRootKey>\solution-index.sqlite
```

What It Stores

- Files: path, hash, length, timestamp, parse status, diagnostic count, stale flag
- Symbols: stable key, path, namespace, type, kind, name, signature, anchor, selector JSON
- References and call sites: target, caller, file, line, column, snippet
- Relationships: partials, inheritance, overrides, interface implementations
- Diagnostics: severity, id, message, line span

What It Answers

- What symbols are in this file?
- What calls this method?
- What references this property, type, field, or event?
- What partials, overrides, and interface relationships matter?
- Is this file stale or diagnostic-bearing?
- What stable key should Claude pass to the next tool?

Boundary: the index is not full MSBuild graph identity yet. It does not model framework/NuGet internals, generated code that only exists during build, or Razor markup semantics. `dotnet build` remains the final correctness gate.

Index-First Read Flow

When Claude already knows the file, it should not guess stable keys or read the whole file first. It should ask the index for the file-local symbol table:

```
query_solution_index(scope: "file", value: "Data/BaseTableRepository.cs")
```

The returned symbol rows contain `StableSymbolKey`, `SourceAnchor`, `SelectorJson`, `Kind`, `Name`, `Signature`, namespace, and containing type.

Then Claude passes the returned stable key to:

```
find_indexed_references(stableSymbolKey)
find_indexed_callers(stableSymbolKey)
find_indexed_relationships(stableSymbolKey)
get_indexed_symbol(stableSymbolKey)
```

Important: stable keys come from index or source-map rows. Claude should not manually compose them from text.

Source Map vs. Solution Index

Surface	Best For	Authority Level
Solution index	Broad navigation, file/symbol discovery, callers, references, relationships, diagnostics	Persisted Roslyn-derived map. Good first signal.
Source map	Live C# file/folder/namespace structure, selector JSON, current file hash	Same-session freshness hop before mutation.
<code>get_symbol</code>	Actual body text for one selected C# symbol	Smallest body read before composing an edit.
<code>get_file</code>	Full file body when unavoidable and below 32KB	Last resort for small files when map/body tools are not enough.

```
index row -> get_source_map(scope: "file", mode: "selector") or check_file_hash ->
get_symbol -> compose candidate
```

Normal Edit Loop

- 1 Claude asks the index for files, symbols, callers, references, and relationships.
- 2 Claude uses a source map to get live selectors and hashes for the file it will edit.
- 3 Claude reads only the needed symbol body with `get_symbol`.
- 4 Claude composes a full candidate into the monitor-owned Working mirror.
- 5 The candidate is staged into an immutable staged record with validation metadata.
- 6 WinMerge or the Host opens a review between watched source and staged candidate.
- 7 The operator accepts the whole candidate or rejects it.
- 8 `record_diff_decision` compares the operator vote with hashes.
- 9 Accepted decisions refresh the solution index so the next query sees current source.

Memory hook

Index to find the right thing, source map to target it safely, Working mirror to compose it, WinMerge to accept it, hashes to classify it.

Multi-File Edits

Coupled C# edits must validate together. This is especially important for async propagation, interface changes, constructor changes, or any signature/API change.

```
start_monitor_session
compose Working candidate A
stage candidate A
```

```
compose Working candidate B
stage candidate B
overlay compile sees A+B together
review A
record decision A
review B
record decision B
record_diff_decision triggers one index rebuild when the chain completes
```

Do not accept file A and then compose file B against the watched-source intermediate state. Compose later candidates against the full proposed Working overlay.

How the overlay is established: pass the same `sessionId` through every candidate composition and `stage_candidate_for_review` call. Each composition tool writes that file's candidate into the monitor-owned `Working\<observedRootKey>\...` mirror. Overlay validation reads the current set of staged/Working candidates for that session together, so file B can compile against file A's proposed candidate before either file is accepted.

Review Gate

The diff is all-or-none. There is no partial hunk merge and no repairing the candidate in WinMerge.

Classification	Meaning
<code>accepted</code>	Operator reported accepted and watched hash equals staged candidate hash.
<code>accepted-normalized</code>	Accepted with only BOM/EOL-normalized differences.
<code>rejected</code>	Operator reported rejected and watched hash equals original baseline hash.
<code>dirty-unexpected</code>	Operator report and watched hash disagree, or watched hash matches neither original nor staged.

Recovery rule: `dirty-unexpected` blocks further staging on that file until explicit refresh/recovery.

Razor and Large Files

Razor is not treated as normal C# symbol surgery.

- Use `replace_text_in_file` for exact small Razor or markup changes.
- Use `replace_span_in_file` when line/column bounds are already known.
- Use `submit_file` for new files, broad rewrites, or unsafe narrow edits.

Cold-session read rule:

- Below 32KB: `get_file` is allowed when index/source-map/symbol context is not enough.
- 32KB and above, or when unsure: call `refresh_file`, then chunk-read the returned Working path.

`refresh_file` does not chunk. It refreshes the Working mirror and returns the local path. The client reads bounded chunks from that path.

Monitor MCP Tool Surface

The Monitor MCP server is the workflow and safety surface. Its tools are grouped by intent.

Group	Representative Tools	What They Do
Status and help	<code>get_monitor_status</code> , <code>get_workflow_status</code> , <code>get_self_check</code> , <code>get_tool_manifest</code> , <code>get_staging_guide</code>	Let a client discover configuration, rules, and debug catalog.
Solution index	<code>query_solution_index</code> , <code>find_indexed_symbols</code> , <code>find_indexed_references</code> , <code>find_indexed_callers</code> , <code>find_indexed_relationships</code>	Read or refresh the SQLite symbol/reference/caller/relationship index.
Session state	<code>start_monitor_session</code> , <code>list_monitor_sessions</code> , <code>get_monitor_session</code> , <code>list_session_staged_records</code>	Give agents durable handles instead of implicit connection memory.
File and structure reads	<code>refresh_file</code> , <code>get_file</code> , <code>check_file_hash</code> , <code>find_file</code> , <code>get_source_map</code> , <code>get_symbol</code>	Read current watched source safely and narrowly.
Candidate composition	<code>submit_file</code> , <code>submit_symbol</code> , <code>replace_text_in_file</code> , <code>replace_span_in_file</code> , <code>add_method</code> , <code>remove_symbol</code>	Write proposed content to the monitor-owned Working mirror only.
Review and decision	<code>stage_candidate_for_review</code> , <code>launch_staged_diff</code> , <code>record_diff_decision</code> , <code>compare_file</code>	Snapshot candidates, launch review, and classify outcomes by vote-plus-hash.
History and cleanup	<code>list_monitor_runs</code> , <code>get_monitor_run</code> , <code>list_ledgers</code> , <code>get_ledger</code> , <code>prune_monitor_history</code>	Inspect monitor history and local ledgers.

Mental model: index tools answer what exists; source-map/symbol tools answer what exact source will be edited; composition tools create the Working candidate; review tools classify the human decision.

Roslyn CodeLens MCP Tool Surface

Roslyn CodeLens MCP is the live compiler-backed semantic analysis surface. It should be used for compiler questions, not mutation.

Tool	Main Argument Shape	Use It For
<code>list_solutions</code>	No special remembered shape	Confirm Roslyn can see the intended solution.
<code>get_diagnostics</code>	<code>project</code> , <code>severity</code> , <code>includeAnalyzers</code>	Compiler warning/error inventory.
<code>search_symbols</code>	<code>query</code>	Discover candidate types and members by name.
<code>get_type_overview</code>	<code>typeName</code>	Confirm namespace, members, dependencies, hierarchy, and file diagnostics.
<code>find_references</code>	<code>symbol</code>	Find references to a type or member. Do not pass <code>symbolName</code> or <code>query</code> .
<code>find_callers</code>	<code>symbol</code>	Find method call sites. Useful for async propagation.
<code>get_call_graph</code>	<code>symbol</code> , <code>direction</code> , <code>maxDepth</code> , <code>maxNodes</code>	Inspect caller/callee chains without manual recursion.
<code>analyze_change_impact</code>	<code>symbol</code>	Estimate blast radius for signature changes, renames, removals, and async conversion.
<code>find_implementations</code>	<code>symbol</code>	Find implementers and derived types, especially for interfaces and virtual/abstract paths.

Common mistake: `search_symbols` uses `query` , but reference/caller/impact tools use `symbol` .

Which MCP Should I Use?

Question	Monitor MCP	Roslyn CodeLens MCP
What files/symbols are in the watched source index?	Yes. This is the cheap first path.	Maybe, but Monitor is cheaper.
What is the stable key for a file-local symbol?	Yes: <code>query_solution_index(scope: "file")</code> .	No.
What calls/references this indexed source symbol?	Yes first: <code>find_indexed_callers/references</code> .	Use as live sanity check or deeper semantic query.
What overrides or implements this symbol?	Yes first: <code>find_indexed_relationships</code> .	Use <code>find_implementations</code> for live compiler perspective.
What exact body should be edited?	Yes: <code>get_source_map</code> then <code>get_symbol</code> .	No mutation authority.
How do I stage/review/accept an edit?	Monitor only.	Never.

Proof and Tests

The important testing idea is that the index is checked against compiler-style known answers.

Fixture Matrix

Generated fixture, Roslyn target counts, Monitor reference/caller/relationship counts. Current matrix passes 68/68.

DBV2 Smokes

Real watched project cross-section and all indexed method/constructor caller verification.

External Corpus

Local sibling repo with `.cs.tst` samples, bind markers, and `expected.json` known answers.

The bind-marker model is the proof: Roslyn is asked what symbol is under `/* <bind>*/.../*</bind>*/`, and Monitor must return matching indexed behavior.

The Mean Teacher End-To-End Test

The best live proof of the full workflow is an async repository conversion.

- 1 Pick one low-fanout repository method.
- 2 Convert it to async.
- 3 Update every actual caller.
- 4 Do not use `.Wait()`, `.Result`, `GetAwaiter().GetResult()`, `Task.Run` sync bridges, or helper bridge wrappers.
- 5 If a caller now awaits, make that caller async and continue propagation until validation is clean.
- 6 Stage all coupled files in one monitor session.

7 Accept/reject through WinMerge.

8 Check `IndexRefresh`.

9 Re-query the index and confirm the async method and callers are visible.

This tests index-first discovery, caller propagation, async reasoning, multi-file staging, overlay compile validation, review gate, automatic index refresh, and post-change query correctness.

WinForms Index Viewer

The `Solution Index` tab is a read-only viewer for the SQLite index. It is not an editor.

- Files grid: what files were indexed and whether they are stale or diagnostic-bearing.
- Symbols grid: declarations, signatures, source anchors, stable keys.
- References tab: incoming references for the selected symbol.
- Callers tab: invocation call sites for methods/constructors and callable symbols.
- Relationships tab: partials, inheritance, overrides, and interface implementation rows.
- Diagnostics tab: persisted diagnostic rows.
- Raw tab: copyable selected symbol data and evidence rows.

If Something Seems Wrong

1 Check the Solution Index UI and `get_solution_index_status`.

2 If file data is stale, run `refresh_solution_index_file` or rebuild the index.

3 Query by file first: `query_solution_index(scope: "file", value: "<relative path>")`.

4 Use the returned `StableSymbolKey`, not a hand-built key.

5 Check references, callers, and relationships.

- 6 If the index missed a real C# symbol/reference, capture the file, symbol, and bind/call site and add a fixture or corpus case.
- 7 Use grep only as a forensic tool after an index/compiler surprise, not as first-pass C# discovery.

Actor rule: Claude may report the evidence and request a new test. Codex or the operator-owned development workflow should change the indexer and test corpus. Claude should not patch indexer internals during a watched-source edit session.